ЧАСТЬ 1. ТЕХНИЧЕСКАЯ ДОКУМЕНТАЦИЯ ПРОЕКТА

Название проекта: Веб-приложение для автоматизации работы ремонтной мастерской «РемМастер».
Цель проекта: Разработка программного продукта для приёма заявок на ремонт техники, отслеживания их статуса, управления складом запчастей и распределения задач между мастерами.

1. Архитектура и стек технологий

Проект реализован по клиент-серверной архитектуре.

Frontend (Клиентская часть): HTML5, CSS3, Vanilla JavaScript (ES6). Для отрисовки графиков статистики используется библиотека Chart.js. Дизайн адаптирован под мобильные устройства (Flexbox, CSS Grid).

Backend (Серверная часть): Node.js, Express.js (REST API).

База данных: MySQL (СУБД). Взаимодействие осуществляется через пул соединений библиотеки mysql2.

Безопасность: Аутентификация и авторизация на основе JWT (JSON Web Tokens). Пароли пользователей хранятся в базе данных в виде хешей (библиотека bcryptjs).

Загрузка файлов: Обработка multipart/form-data с помощью библиотеки multer для сохранения фотографий неисправностей на сервере.

2. Структура базы данных (MySQL)

База данных repair_shop состоит из следующих ключевых таблиц:

users — хранит информацию о пользователях. Роли: client (клиент), master (мастер), admin (администратор).

services — каталог предоставляемых услуг с описанием и базовой ценой.

requests — главная таблица заявок. Содержит внешние ключи на клиента (client_id) и мастера (master_id), статус заявки, описание, фото, итоговую стоимость и комментарии мастера.

inventory — склад запчастей (название, количество, цена).

3. Роли пользователей и функционал

3.1. Клиент (Client)

Просмотр каталога услуг на главной странице.

Регистрация и авторизация в системе.

Создание заявки на ремонт с возможностью прикрепления фотографии устройства.

Просмотр истории своих обращений и отслеживание актуального статуса ремонта в личном кабинете.

3.2. Мастер (Master)

Авторизация в рабочем кабинете.

Просмотр списка заявок, назначенных администратором.

Изменение статуса заявки (Принята, Диагностика, В ремонте, Готова, Выдана).

Добавление комментариев к заявке (видны клиенту).

Указание использованных запчастей и выставление итоговой стоимости работ.

3.3. Администратор (Admin)

Панель управления с разбивкой по вкладкам.

Назначение свободных мастеров на новые заявки.

Управление складом запчастей (добавление новых позиций).

Просмотр списка сотрудников.

Просмотр статистики: графики распределения заявок по статусам и график доходов по месяцам.

4. Основные маршруты REST API

POST /api/auth/login — авторизация пользователя (выдача JWT).

POST /api/auth/register — регистрация нового клиента.

GET /api/services — получение списка услуг.

POST /api/requests — создание новой заявки (с загрузкой фото).

GET /api/requests/my — получение заявок текущего клиента.

PATCH /api/requests/:id — обновление заявки мастером.

POST /api/requests/:id/assign — назначение мастера (только админ).

GET /api/inventory — получение остатков на складе.

GET /api/stats — получение данных для графиков (только админ).

ЧАСТЬ 2. ОТЧЁТ ПО ТЕСТИРОВАНИЮ (ЧЕК-ЛИСТ)

Объект тестирования: Веб-приложение «РемМастер» (Frontend + Backend + БД).
Окружение тестирования: ОС Windows 11, Node.js v20+, MySQL 8.0, Браузер Google Chrome (v120+), Инструменты разработчика (DevTools).

Метод тестирования: Функциональное тестирование (Black Box), тестирование API (Postman), проверка граничных значений.

Таблица результатов тестирования

ID, Модуль / Функция, Тестовый сценарий (Действие), Ожидаемый результат, Статус

1. Авторизация и доступ, , , ,

1.1, Вход в систему, Ввод валидных данных клиента (client@test.ru / 123456), Успешный вход, редирект в кабинет клиента, сохранение JWT в LocalStorage, Пройдено

1.2, Вход в систему, Ввод неверного пароля, Вывод сообщения об ошибке, доступ запрещен, Пройдено

1.3, Защита роутов, Попытка зайти на admin-dashboard.html без токена или под ролью клиента, Редирект на страницу авторизации, вывод сообщения "Доступ запрещен", Пройдено

1.4, Регистрация, Создание нового аккаунта (валидные данные), Запись в БД, хеширование пароля, редирект на страницу входа, Пройдено

2. Кабинет Клиента, , , ,

2.1, Просмотр услуг, Открытие главной страницы, Успешная загрузка карточек услуг с ценами из БД, Пройдено

2.2, Создание заявки, Заполнение формы, выбор услуги, ввод описания (>10 символов), Заявка создается, появляется в таблице "Мои заявки" со статусом "Принята", Пройдено

2.3, Создание заявки, Ввод описания менее 10 символов, Валидация на фронтенде, вывод предупреждения, заявка не уходит на сервер, Пройдено

2.4, Загрузка фото, Прикрепление изображения (jpg/png) к заявке, Файл сохраняется в папку public/uploads, путь записывается в БД, Пройдено

3. Кабинет Мастера, , , ,

3.1, Просмотр заявок, Авторизация под мастером (master@test.ru), Загрузка только тех заявок, которые назначены на этого мастера, Пройдено

3.2, Обновление статуса, Открытие модального окна, смена статуса на "В ремонте", сохранение, Данные в БД обновляются, статус меняется в таблице без перезагрузки, Пройдено

3.3, Комментарии, Ввод комментария и стоимости работ в модальном окне, Комментарий сохраняется в БД, стоимость обновляется в карточке клиента, Пройдено

4. Кабинет Администратора, , , ,

4.1, Назначение мастера, Нажатие кнопки "Назначить", выбор мастера из списка, Запрос POST /assign проходит успешно, имя мастера появляется в таблице, Пройдено

4.2, Управление складом, Добавление новой запчасти (название, кол-во, цена), Запись добавляется в БД, таблица склада обновляется, Пройдено

4.3, Статистика, Открытие вкладки "Статистика", Библиотека Chart.js корректно отрисовывает данные из БД (круговая и линейная диаграммы), Пройдено

5. Нефункциональные требования, , , ,

5.1, Адаптивность, Открытие сайта на мобильном устройстве (эмуляция в Chrome), Интерфейс перестраивается, элементы не наезжают друг на друга, Пройдено

5.2, Безопасность БД, Проверка SQL-инъекций через формы ввода, Запросы используют параметризацию (?), инъекции невозможны, Пройдено

Заключение по тестированию

Все ключевые пользовательские сценарии (создание заявки, её обработка мастером, назначение админом) работают корректно. Обработка ошибок на стороне клиента и сервера функционирует. Взаимодействие с базой данных стабильно, утечек данных и ошибок CORS не выявлено. Приложение готово к демонстрации и защите проекта.